WEEK 10: ADVANCED SQL QUERY SYSTEM SCENARIO

Scenario:

The ArtWork Shop wants to perform advanced analysis on customer purchases,product pricing,and sales performance.The management team needs complex reports to identify high-value customers,expensive products, and business trends. It will implement subqueries and nested queries to extract deeper insights from the Artwork Shop Database.

Tasks to be Done:

1. Implementation of Subqueries and Nested Queries

Subqueries and Nested Queries are SQL techniques used to retrieve information by executing one query within another query. They help perform advanced database operations by using the output of an inner query to obtain the required results in the outer query.

Types Covered:

- Single-Row Subquery : Returns one row.
- Multi-Row Subquery : Returns more than one row.
- Nested Subquery : A query written inside another query.

2. Find Products Above Average Price Task:

1. Find products priced above the average price.

```
SQL> SELECT AVG(PRICE) FROM PRODUCT;

AVG(PRICE)
----------
3329.13333

SQL> SELECT PRODUCT_NAME,PRICE FROM PRODUCT WHERE PRICE>(SELECT AVG(PRICE) FROM PRODUCT);

PRODUCT_NAME              PRICE
-------------------- ----------
Smart phone               24999
Office Chair               5499
Study Table                6999
```

2.Display product name, category, and price.

```
SQL> SELECT PRODUCT_NAME, CATEGORY_ID, PRICE FROM PRODUC
T;

PRODUCT_NAME         CATEGORY_ID      PRICE
-------------------- ----------- ----------
Smart phone                    1      24999
Wireless Mouse                 1        799
Bluetooth Speaker              1       1499
Cotton Shirt                   2        899
Denim Jeans                    2       1599
Data Structures                3        650
Python programming             3        850
Electric Kettle                4       1299
Running Shoes                  5       2199
Office Chair                   9       5499
Building Blocks               10        999

PRODUCT_NAME         CATEGORY_ID      PRICE
-------------------- ----------- ----------
Cooking oil                    7        699
Face wash                      6        349
Study Table                    9       6999
Yoga Mat                       5        599

15 rows selected.
```

3.Identify expensive products in each category.

```
SQL> SELECT P1.PRODUCT_NAME, P1.CATEGORY_ID,P1.PRICE FRO
M PRODUCT P1
  2  WHERE P1.PRICE >(SELECT AVG(P2.PRICE)
  3  FROM PRODUCT P2
  4  WHERE P1.CATEGORY_ID=P2.CATEGORY_ID);

PRODUCT_NAME         CATEGORY_ID      PRICE
-------------------- ----------- ----------
Smart phone                    1      24999
Denim Jeans                    2       1599
Python programming             3        850
Running Shoes                  5       2199
Study Table                    9       6999
```

4. Find products with price higher than a selected category average.

```
SQL> SELECT PRODUCT_NAME, CATEGORY_ID, PRICE
  2  FROM PRODUCT
  3  WHERE CATEGORY_ID=9
  4  AND PRICE>(SELECT AVG(PRICE)
  5  FROM PRODUCT
  6  WHERE CATEGORY_ID =9);

PRODUCT_NAME         CATEGORY_ID      PRICE
-------------------- ----------- ----------
Study Table                    9       6999
```

3. Identify Customers with Maximum Purchases Task:

1. Customer who spent the highest amount.

```
SQL> SELECT CUSTOMER_ID,TOTAL_AMOUNT
  2  FROM ORDERS
  3  WHERE TOTAL_AMOUNT=(SELECT MAX(TOTAL_AMOUNT) FROM O
RDERS);

CUSTOMER_ID TOTAL_AMOUNT
----------- ------------
          1         5100
```

2. Customers with maximum number of orders.

```
SQL> SELECT CUSTOMER_ID, COUNT(*) AS NO_OF_ORDERS
  2  FROM ORDERS
  3  GROUP BY CUSTOMER_ID
  4  HAVING COUNT(*)=(
  5  SELECT MAX(NO_OF_ORDERS)
  6  FROM (
  7  SELECT COUNT(*) AS NO_OF_ORDERS
  8  FROM ORDERS
  9  GROUP BY CUSTOMER_ID));

CUSTOMER_ID NO_OF_ORDERS
----------- ------------
          1            3
```

3. Customers whose spending is above average.

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) AS TOTAL_SPEND
ING
  2  FROM ORDERS
  3  GROUP BY CUSTOMER_ID
  4  HAVING SUM(TOTAL_AMOUNT)>(
  5  SELECT AVG(TOTAL_SPENDING)
  6  FROM (
  7  SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  8  FROM ORDERS
  9  GROUP BY CUSTOMER_ID));

CUSTOMER_ID TOTAL_SPENDING
----------- --------------
          1          10800
          2           4550
          4           5800
          3           4900
         10           4500
```

4. Top 5 valuable customers.

```
SQL> SELECT * FROM(SELECT CUSTOMER_ID, SUM(TOTAL_AMOUNT)
 AS TOTAL_SPENDING FROM ORDERS
  2  GROUP BY CUSTOMER_ID
  3  ORDER BY TOTAL_SPENDING DESC)
  4  WHERE ROWNUM <=5;

CUSTOMER_ID TOTAL_SPENDING
----------- --------------
          1          10800
          4           5800
          3           4900
          2           4550
         10           4500
```

4. Generate Complex Business Queries

Query 1: Best-Selling Product

Find products with maximum sales quantity. Tasks:

- Display product name.
- Total quantity sold.
- Total revenue generated.

```
SQL> SELECT P.PRODUCT_NAME
  2  ,SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD,
  3  SUM(OD.QUANTITY*OD.PRICE) AS TOTAL_REVENUE
  4  FROM PRODUCT P, ORDER_DETAILS OD
  5  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  6  GROUP BY P.PRODUCT_ID,P.PRODUCT_NAME
  7  HAVING SUM(OD.QUANTITY)=(SELECT MAX(SUM(QUANTITY))
  8  FROM ORDER_DETAILS
  9  GROUP BY PRODUCT_ID);

PRODUCT_NAME         TOTAL_QUANTITY_SOLD TOTAL_REVENUE
-------------------- ------------------- -------------
Running Shoes                          3          5100
Cotton Shirt                           3          1500
```

Query 2: High-Value Customers

Find customers whose total purchase amount is greater than average customer spending

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) AS TOTAL_SPENT

  2  FROM ORDERS
  3  GROUP BY CUSTOMER_ID
  4  HAVING SUM(TOTAL_AMOUNT)>(SELECT AVG(SUM(TOTAL_AMOU
NT))
  5  FROM ORDERS
  6  GROUP BY CUSTOMER_ID);

CUSTOMER_ID TOTAL_SPENT
----------- -----------
          1       10800
          2        4550
          4        5800
          3        4900
         10        4500
```

Query 3: Category Performance Analysis

Find:

- Highest revenue-generating category.
- Number of products sold in each category.
- Average product price by category.

```
SQL> SELECT C.CATEGORY_ID,
  2  SUM(OD.QUANTITY*OD.PRICE) AS TOTAL_REVENUE,
  3  SUM(OD.QUANTITY) AS TOTAL_PRODUCT_SOLD,
  4  AVG(P.PRICE) AS AVERAGE_PRODUCT_PRICE
  5  FROM CATEGORY C, PRODUCT P, ORDER_DETAILS OD
  6  WHERE C.CATEGORY_ID=P.CATEGORY_ID
  7  AND P.PRODUCT_ID=OD.PRODUCT_ID
  8  GROUP BY C.CATEGORY_ID
  9  ORDER BY TOTAL_REVENUE DESC;

CATEGORY_ID TOTAL_REVENUE TOTAL_PRODUCT_SOLD AVERAGE_PRODUCT_PRICE
----------- ------------- ------------------ ---------------------
          1          7500                  4                  9099
          9          6800                  2                  6249
          5          6700                  5                  1399
          3          6350                  3                   750
          2          5700                  5                  1249
          6          4500                  1                   349
          7          3400                  1                   699
         10          1900                  2                   999
          4          1200                  1                  1299

9 rows selected.
```

Query 4: Customer Purchase History

Display:

- Customer name
- Number of orders
- Total purchase amount
- Most purchased product

```
SQL> SELECT C.CUSTOMER_NAME, COUNT(O.ORDER_ID), SUM(O.TOTAL_AMOUNT),
  2  (SELECT P.PRODUCT_NAME FROM PRODUCT P,ORDER_DETAILS OD
  3  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  4  AND OD.ORDER_ID=O.ORDER_ID AND ROWNUM=1)
  5  FROM CUSTOMER C, ORDERS O
  6  WHERE C.CUSTOMER_ID=O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME,O.ORDER_ID;

CUSTOMER_NAME                                      COUNT(O.ORDER_ID) SUM(O.TOTAL_AMOUNT) (SELECTP.PRODUCT_NAM
-------------------------------------------------- ----------------- ------------------- --------------------
Rahul Raj                                                          1                1500 Cotton Shirt
Karthik M                                                          1                4200 Denim Jeans
Anjali Devi                                                        1                3600 Python programming
Arun Kumar                                                         1                5100 Running Shoes
Priya Sharma                                                       1                1800 Wireless Mouse
Arun Kumar                                                         1                3200 Bluetooth Speaker
Priya Sharma                                                       1                2750 Data Structures
Nithya S                                                           1                4500 Study Table
Vijay Kumar                                                        1                1200 Electric Kettle
Karthik M                                                          1                1600 Yoga Mat
Arun Kumar                                                         1                2500 Smart phone
Ajay Kumar                                                         1                1900 Building Blocks
Rahul Raj                                                          1                3400 Cooking oil
Meeena K                                                           1                2300 Office Chair
Manoj R                                                            1                2800 Face wash

15 rows selected.
```

5. Prepare Advanced SQL Report:

Report 1: Premium Product Report

Display:

- Product Name
- Category
- Price
- Stock Availability Condition:

Products above average price.

```
SQL>  SELECT P.PRODUCT_NAME, C.DESCRIPTION, P.PRICE,P.STOCK_QUANTITY
  2  FROM PRODUCT P, CATEGORY C
  3  WHERE P.CATEGORY_ID=C.CATEGORY_ID
  4  AND P.PRICE>(SELECT AVG(PRICE) FROM PRODUCT);

PRODUCT_NAME
--------------------
DESCRIPTION
--------------------------------------------------------------------------------
--------------
     PRICE STOCK_QUANTITY
---------- --------------
Smart phone
Electronic devices and accessories
     24999             50

Office Chair
Home and office furniture
      5499             15

Study Table
Home and office furniture
      6999             10
```

Report 2: Customer Value Report

Display:

- Customer Name
- Total Orders
- Total Spending
- Customer Category

```
SQL> SELECT C.CUSTOMER_NAME,COUNT(O.ORDER_ID) AS TOTAL_ORDERS,
  2  SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING,
  3  CASE
  4  WHEN SUM(O.TOTAL_AMOUNT)>=100000 THEN 'PREMIUM'
  5  WHEN SUM(O.TOTAL_AMOUNT)>=50000 THEN 'REGULAR'
  6  ELSE 'BASIC'
  7  END AS CUSTOMER_CATEGORY
  8  FROM CUSTOMER C, ORDERS O
  9  WHERE C.CUSTOMER_ID=O.CUSTOMER_ID
 10  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME;

CUSTOMER_NAME                                      TOTAL_ORDERS TOTAL_SPENDING CUSTOME
-------------------------------------------------- ------------ -------------- -------
Arun Kumar                                                    3  10800 BASIC
Vijay Kumar                                                   1   1200 BASIC
Priya Sharma                                                  2   4550 BASIC
Rahul Raj                                                     2   4900 BASIC
Meeena K                                                      1   2300 BASIC
Karthik M                                                     2   5800 BASIC
Anjali Devi                                                   1   3600 BASIC
Nithya S                                                      1   4500 BASIC
Ajay Kumar                                                    1   1900 BASIC
Manoj R                                                       1   2800 BASIC

10 rows selected.
```

Report 3: Sales Performance Report

Display:

- Product-wise sales

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_SALES
  2  FROM PRODUCT P, ORDER_DETAILS OD
  3  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  4  GROUP BY P.PRODUCT_NAME;

PRODUCT_NAME         TOTAL_SALES
-------------------- -----------
Data Structures                1
Cooking oil                    1
Yoga Mat                       2
Smart phone                    1
Face wash                      1
Cotton Shirt                   3
Office Chair                   1
Building Blocks                2
Wireless Mouse                 2
Bluetooth Speaker              1
Denim Jeans                    2
Electric Kettle                1
Python programming             2
Running Shoes                  3
Study Table                    1

15 rows selected.
```

- Category-wise revenue

```
SQL> SELECT C.DESCRIPTION AS CATEGORY, SUM(OD.QUANTITY*OD.PRICE) AS REVENUE
  2  FROM CATEGORY C, PRODUCT P, ORDER_DETAILS OD
  3  WHERE C.CATEGORY_ID=P.CATEGORY_ID
  4  AND P.PRODUCT_ID=OD.PRODUCT_ID
  5  GROUP BY C.DESCRIPTION;

CATEGORY
--------------------------------------------------------------------------------
   REVENUE
----------
Men and women clothing products
      5700

Sports and fitness equipment
      6700

Home and office furniture
      6800

Daily grocery and food products
      3400

Beauty and personal care products
      4500

Appliances used at home
      1200

Toys and games for children
      1900

Electronic devices and accessories
      7500

Educational and general books
      6350


9 rows selected.
```

- Top-selling products

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_SALES
  2  FROM PRODUCT P, ORDER_DETAILS OD
  3  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  4  GROUP BY P.PRODUCT_NAME
  5  HAVING SUM(OD.QUANTITY)=(SELECT MAX(SUM(QUANTITY))
  6  FROM ORDER_DETAILS
  7  GROUP BY PRODUCT_ID);

PRODUCT_NAME         TOTAL_SALES
-------------------- -----------
Cotton Shirt                   3
Running Shoes                  3
```

Report 4: Business Decision Report

Identify:

● High-performing products

```
SQL> SELECT P.PRODUCT_NAME,SUM(OD.QUANTITY) AS TOTAL_SALES
  2  FROM PRODUCT P, ORDER_DETAILS OD
  3  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  4  GROUP BY P.PRODUCT_NAME
  5  HAVING SUM(OD.QUANTITY)=(SELECT MAX(SUM(QUANTITY))
  6  FROM ORDER_DETAILS GROUP BY PRODUCT_ID);

PRODUCT_NAME         TOTAL_SALES
-------------------- -----------
Cotton Shirt                   3
Running Shoes                  3
```

● Low-performing products

```
SQL> SELECT P.PRODUCT_NAME,SUM(OD.QUANTITY) AS TOTAL_SALES
  2  FROM PRODUCT P,ORDER_DETAILS OD
  3  WHERE P.PRODUCT_ID=OD.PRODUCT_ID
  4  GROUP BY P.PRODUCT_NAME
  5  HAVING SUM(OD.QUANTITY)=(SELECT MIN(SUM(QUANTITY))
  6  FROM ORDER_DETAILS GROUP BY PRODUCT_ID);

PRODUCT_NAME         TOTAL_SALES
-------------------- -----------
Data Structures                1
Cooking oil                    1
Smart phone                    1
Face wash                      1
Office Chair                   1
Bluetooth Speaker              1
Electric Kettle                1
Study Table                    1

8 rows selected.
```

● High-value customers

```
SQL> SELECT CUSTOMER_ID,SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
  2  FROM ORDERS
  3  GROUP BY CUSTOMER_ID
  4  HAVING SUM(TOTAL_AMOUNT)>(SELECT AVG(SUM(TOTAL_AMOUNT))
  5  FROM ORDERS GROUP BY CUSTOMER_ID);

CUSTOMER_ID TOTAL_SPENDING
----------- --------------
          1          10800
          2           4550
          4           5800
          3           4900
         10           4500
```